Отчет

по лабораторной работе № 3

Объектное хранилище RustFS, MapReduce и DuckDB

Выполнила: Лорс Алина, ИСУ: 474321

Группа: ИИИ-3-ИнжДан-1-5

Вариант 9: Программист, разработчик на мобильных платформах

1. Цель работы и исходные данные

развернуть локальное S3-совместимое хранилище RustFS и независимо подсчитать частоты одиночных навыков, пар и троек навыков двумя способами, собственной реализацией MapReduce и SQL-запросами DuckDB. Затем сравнить результаты, время выполнения, потребление памяти, зарплаты и три региона.

Фильтр варианта: роль «Программист, разработчик» и маркер мобильной разработки в названии (Android, iOS, Flutter, Mobile, слово с основой «мобильн», React Native либо Kotlin Multiplatform). Один и тот же фильтр реализован отдельно в пайтон и SQL

2. Развёртывание RustFS

RustFS запущен через Docker Compose. Сервис permissions выполняет настройку прав на тома, затем запускается rustfs. Консоль доступна по адресу http://localhost:9001, S3 API == http://localhost:9000. Для учебного эксперимента использован бакет hh-lake.

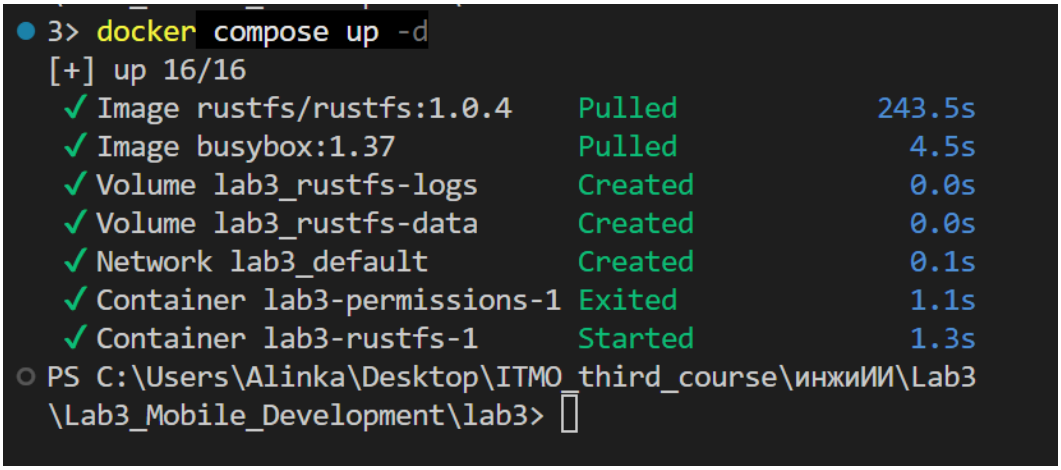


Рисунок 1.Запуск сервисов через docker compose up -d

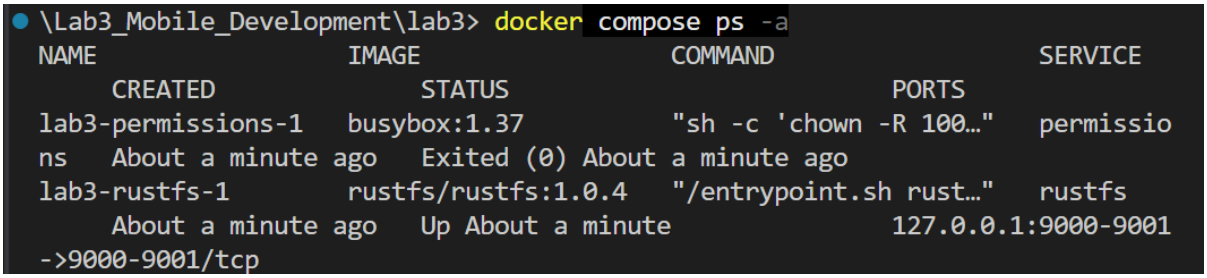


Рисунок 2. Проверка статуса docker compose ps -a

3. Подсчёт навыков двумя независимыми способами

В MapReduce CSV читается из RustFS порциями по 2 000 записей. Два mapper-процесса строят счётчики, reducer суммирует результаты, одновременно в очереди находится не более четырёх задач. В DuckDB SQL-запросы обращаются к тому же объекту (через расширение httpfs)ю

Для каждой вакансии навыки приводятся к нижнему регистру, очищаются от пробелов и дубликатов. Частота набора это число вакансий, в которых одновременно встречаются все навыки этого набора.

Размер набора	Различных наборов	Сумма частот
Один навык	1 007	14 904
Пара	11 950	46 696
Тройка	59 871	123 452

Число сочетаний растёт: при m уникальных навыках одна вакансия даёт m одиночных навыков, m(m−1)/2 пар и m(m−1)(m−2)/6 троек. Поэтому общая сумма частот троек больше, чем у одиночных навыков.

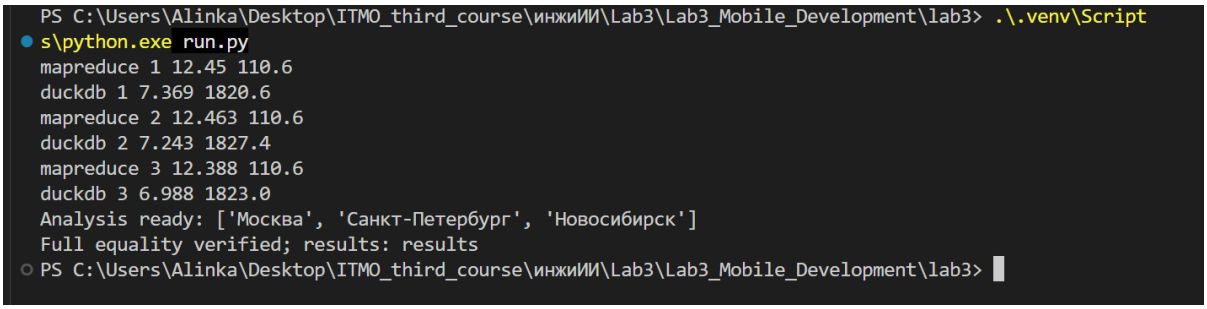


Рисунок 3. Успешный запуск трёх попыток расчёта

4. Результаты в объектном хранилище

Исходный CSV хранится в hh-lake/it_vacancies_2024.csv. После расчёта создан префикс hh-lake/lab3/ с файлами результатов, статистикой, логами, профилями и региональным анализом.

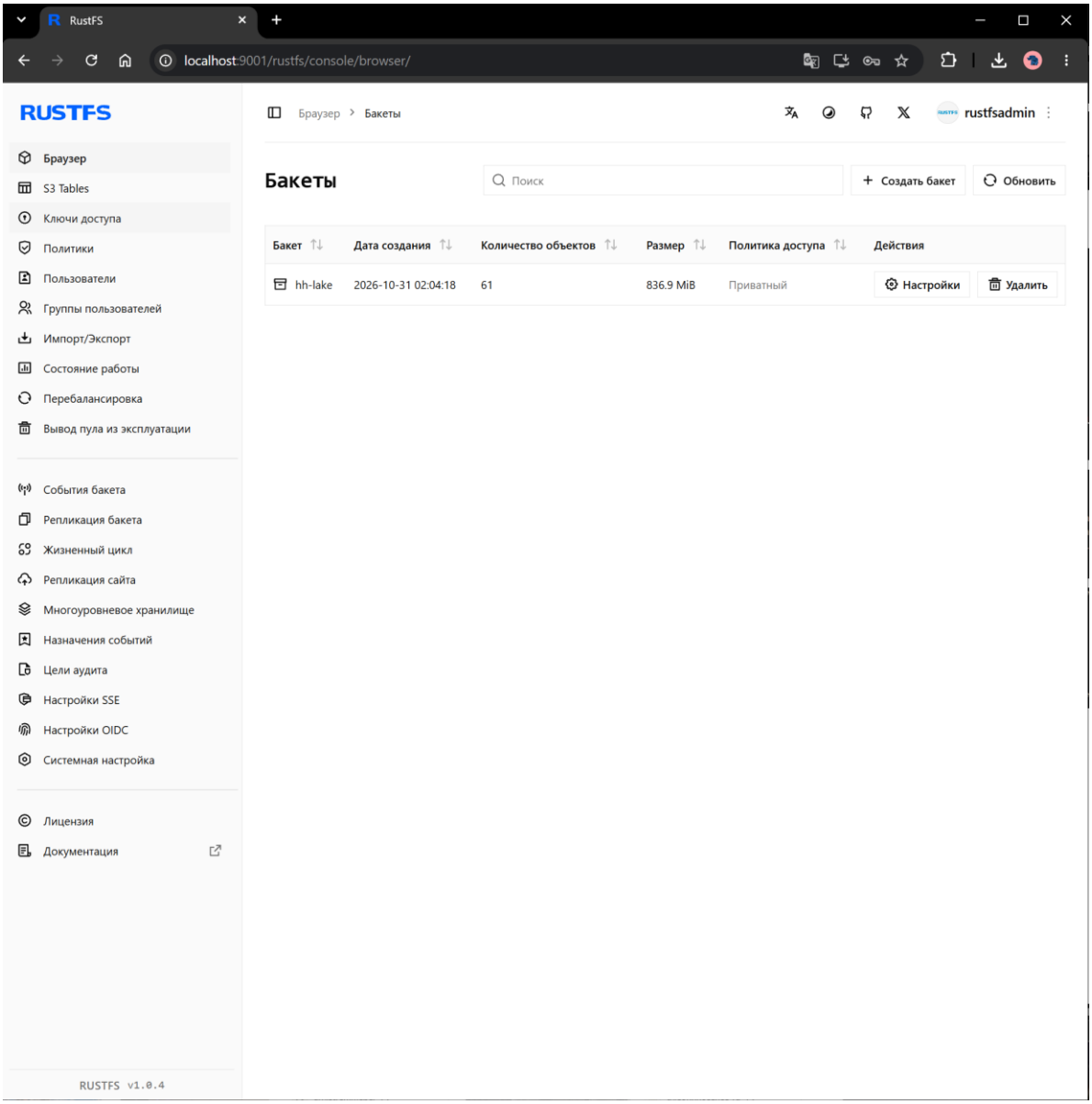


Рисунок 4. Бакет hh-lake в интерфейсе RustFS

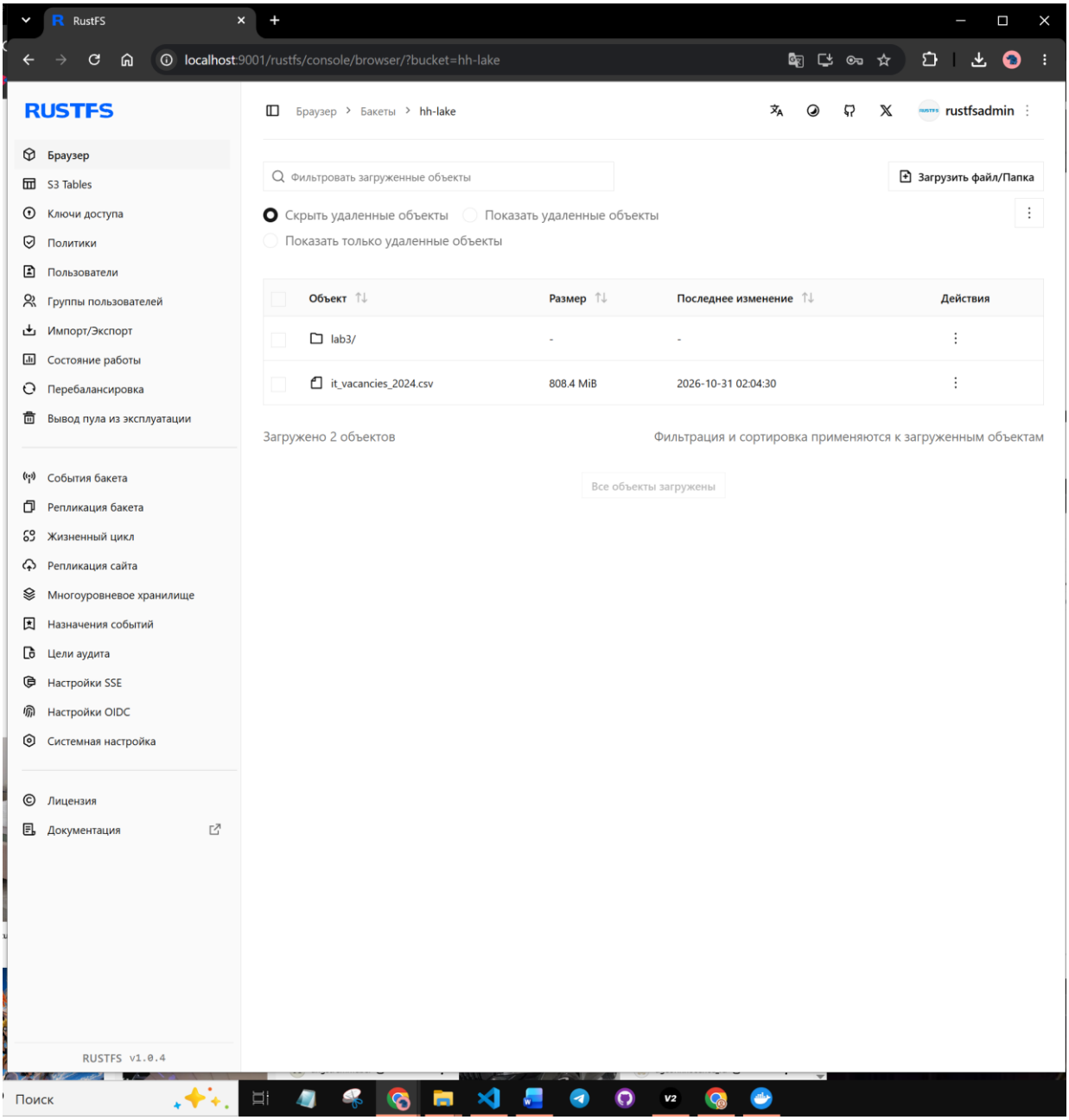


Рисунок 5. Исходный CSV и каталог lab3 в бакете

5. Проверка эквивалентности MapReduce и DuckDB

Каждый подход запускался три раза. Для k = 1, 2 и 3 после каждой попытки программа проверяла полное равенство словарей ключ, частота между MapReduce и DuckDB. Во всех девяти попарных сравнениях расхождений не обнаружено, еще дополнительно повторные запуски MapReduce проверены относительно первого.

Отдельно я проверила обработка исключения в одной задаче mapper: обработка этой порции повторяется, а результат учитывается один раз. По файлам verification.json и failure/summary.json зарегистрирована одна повторная попытка (retries = 1), а итоговые три таблицы еще и совпали с эталонными.

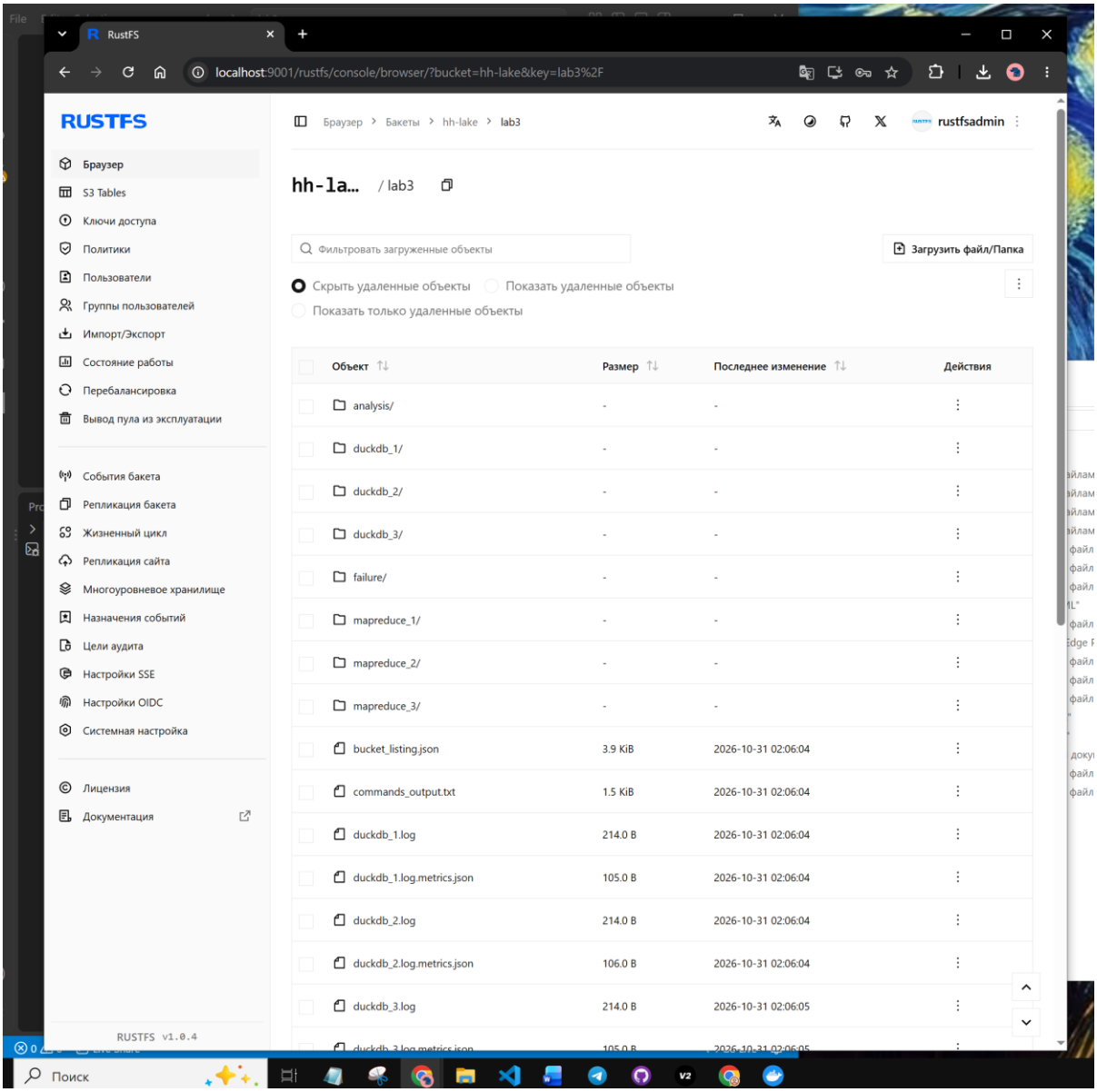


Рисунок 6. Каталоги трёх запусков MapReduce и DuckDB в hh-lake/lab3

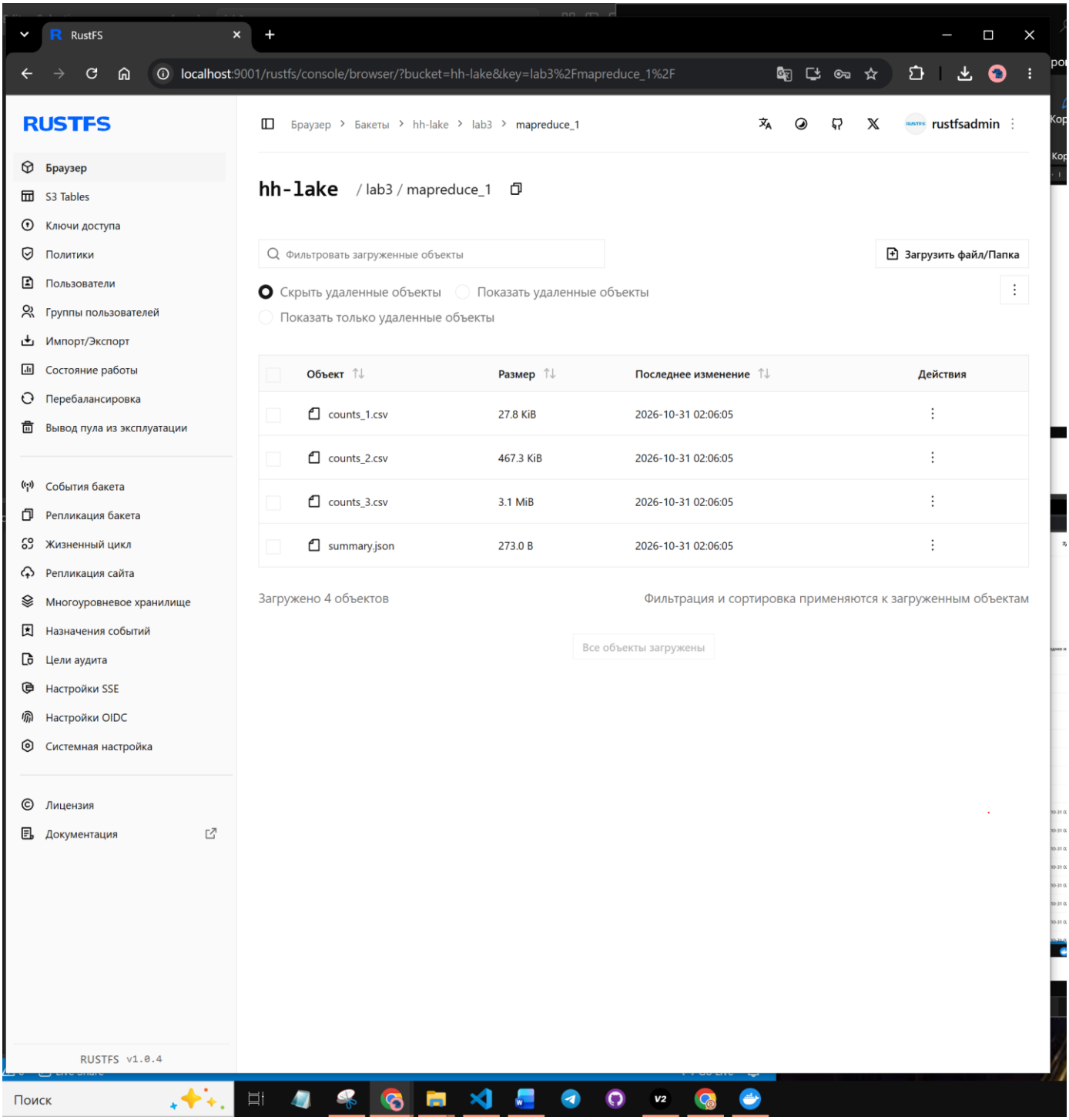


Рисунок 7.Файлы counts_1.csv, counts_2.csv, counts_3.csv первого запуска MapReduce

6. Сравнение времени и памяти

Время включает запуск измеряемой программы, чтение из S3, расчёт и локальную запись результатов. Пик памяти то есть периодическая сумма RSS дерева процессов, включая дочерние процессы; память сервера RustFS не включалась.

Метод	Попытка	Время, с	Пик RSS, МиБ
MapReduce	1	12,450	110,6
DuckDB	1	7,369	1820,6
MapReduce	2	12,463	110,6
DuckDB	2	7,243	1827,4
MapReduce	3	12,388	110,6
DuckDB	3	6,988	1823,0

Среднее время MapReduce = 12,43 с, DuckDB = 7,20 с, В данной конфигурации DuckDB быстрее примерно в 1,73 раза, но использует в среднем 1823,7 МиБ против 110,6 МиБ у MapReduce (около 16,5 раза больше),

Разница связана с разным способом обработки:

MapReduce накапливает агрегаты по небольшим порциям CSV, а DuckDB формирует внутренние структуры для SQL-вычислений. Важно отметить, что полученные значения относятся к конкретной машине и настройкам, а не к универсальному превосходству одного метода.

Наиболее частые одиночные навыки

Навык	Вакансий
Android	1 163
Kotlin	1 066
iOS	890
Git	817
Swift	704
Android SDK	605
Java	595
Flutter	441

7. Анализ зарплат и сочетаний навыков

Для сравнения использованы только вакансии с зарплатой в рублях (RUR) и gross=false. Если указаны обе границы, бралась середина вилки. Другие валюты и зарплаты до удержаний я исключила. Сопоставимых вакансий = 973 из 3 713.

Для пары базой является более частый из двух отдельных навыков, для тройки частый это наиболее частая вложенная пара. Частота базового набора измеряется по всей

отфильтрованной выборке, а не только по зарплатным строкам. В итоговый анализ включаются сочетания и базы как минимум с 10 зарплатными наблюдениями.

Пары навыков

Сочетание	n с з/п	Медиана, ₽	База (n с з/п)	Медиана базы, ₽	Разница
Coroutines + MVVM	10	270 000	MVVM (n=84)	132 500	+103,8 %
Android SDK + Hilt	11	250 000	Android SDK (n=151)	150 000	+66,7 %
Android Studio + Kotlin	12	92 500	Kotlin (n=256)	190 000	-51,3 %
json api + rest	10	75 000	rest (n=72)	150 000	-50,0 %

Тройки навыков

Сочетание	n с з/п	Медиана, ₽	База (n с з/п)	Медиана базы, ₽	Разница
Coroutines + Kotlin + MVVM	10	270 000	Kotlin + MVVM (n=56)	150 000	+80,0 %
Android + MVI + MVVM	11	250 000	Android + MVVM (n=43)	140 000	+78,6 %
Android + Android Studio + Kotlin	10	92 500	Android + Kotlin (n=201)	190 000	-51,3 %
Android + Git + PostgreSQL	11	80 000	Android + Git (n=132)	150 000	-46,7 %

Например, у пары Coroutines + MVVM медиана 270 000 ₽ (10 вакансий с зарплатой) против 132 500 ₽ у MVVM (84 наблюдения), разница +103,8 %. Для тройки Coroutines + Kotlin + MVVM медиана 270 000 ₽ (10 наблюдений) против 150 000 ₽ у базовой пары Kotlin + MVVM (56 наблюдений), разница +80,0 %.

Отрицательные примеры:

Android Studio + Kotlin == 92 500 ₽ против 190 000 ₽ у Kotlin (−51,3 %); Android + Android Studio + Kotlin == 92 500 ₽ против 190 000 ₽ у пары Android + Kotlin (−51,3 %). Но различия не доказывают причинного влияния навыка: группы могут различаться по опыту, региону и работодателю, а выборки отдельных комбинаций невелики.

Источник чисел: results/analysis/pair_effects.csv и triple_effects.csv. n с з/п. число вакансий, использованных для медианы конкретного сочетания.

8. Региональный анализ

Для сравнения выбраны три региона с наибольшим количеством вакансий варианта: Москва, Санкт-Петербург и Новосибирск. Зарплатные медианы рассчитаны только по тем же сопоставимым зарплатам в RUR при gross=false.

Регион	Всего вакансий	С зарплатой	Медиана, ₽	Среднее навыков
Москва	1 735	403	200 000	3,84
Санкт-Петербург	581	147	160 000	3,69
Новосибирск	138	34	122 500	4,93

Медиана в Москве = 200 000 ₽, в Санкт-Петербурге = 160 000 ₽, в Новосибирске = 122 500 ₽. В Новосибирске среднее число навыков больше (4,93 против 3,84 и 3,69), но там всего 34 вакансии с сопоставимой зарплатой; результаты следует интерпретировать осторожно.

Сравнение востребованных навыков, пар и троек

Ниже приведены два наиболее частых набора каждого размера в каждом регионе. После названия в скобках указано число вакансий с этим набором. Полные таблицы, включая число зарплатных наблюдений и медианы по наборам, сохранены в region_stacks_1.csv, region_stacks_2.csv и region_stacks_3.csv.

Одиночные навыки

Регион	1-й по частоте	2-й по частоте
Москва	Kotlin (488)	Android (483)
Санкт-Петербург	Android (175)	Kotlin (170)
Новосибирск	Android (42)	Kotlin (42)

Пары навыков

Регион	1-й по частоте	2-й по частоте
Москва	Android + Kotlin (342)	iOS + Swift (244)
Санкт-Петербург	Android + Kotlin (129)	iOS + Swift (90)
Новосибирск	Android + Kotlin (24)	iOS + Swift (24)

Тройки навыков

Регион	1-й по частоте	2-й по частоте
Москва	Android + Android SDK + Kotlin (159)	Android + Java + Kotlin (146)
Санкт-Петербург	Android + Java + Kotlin (67)	Android + Android SDK + Kotlin (49)
Новосибирск	Android + Android SDK + Kotlin (10)	Android + Git + Kotlin (10)

В Москве среди одиночных навыков лидируют Kotlin (488) и Android (483), а в Санкт-Петербурге Android (175) и Kotlin (170). В Новосибирске эти два навыка делят первое место (по 42 вакансии). Поэтому формулировка «лидер» для Новосибирска не означает уникального победителя.

Самая частая пара во всех трёх регионах Android + Kotlin: 342 вакансии в Москве, 129 в Санкт-Петербурге, 24 в Новосибирске. В Москве это 19,7 % всех отобранных вакансий региона, в Санкт-Петербурге 22,2 %, в Новосибирске 17,4 %. Абсолютные частоты нельзя сравнивать без учёта различий в общем числе вакансий.

Среди троек Москва выделяется сочетанием Android + Android SDK + Kotlin (159), а Санкт-Петербург Android + Java + Kotlin (67). В Новосибирске пять троек имеют по 10 вакансий; первые две строки таблицы выбраны по вторичному алфавитному порядку, а не из-за отличия в частоте. Малое число региональных наблюдений ограничивает выводы о различиях рынка.

Источник: results/analysis/regions.csv и region_stacks_1.csv, region_stacks_2.csv, region_stacks_3.csv. Региональные таблицы построены по всем вакансиям варианта, а не только по 973 вакансиям с зарплатой.

9. Итоги

Я получила одинаковые результаты двумя независимыми способами: 1 007 одиночных навыков, 11 950 пар и 59 871 троек, с полным совпадением частот во всех трёх попытках. Повторная обработка ошибочной задачи mapper не изменила итоговые таблицы.DuckDB на данном компьютере выполнил расчёт быстрее, но занял значительно больше оперативной памяти. В зарплатном анализе обнаружены сочетания с более высокой и более низкой медианой относительно частого базового набора; причинный эффект из этих данных не следует.

Региональное сравнение показало различия медианных зарплат и востребованных технологических сочетаний. Для наглядности в отчёт включены не только вакансии и зарплаты, но и наиболее частые одиночные навыки, пары и тройки по каждому региону.